Lecture 1: Introduction & C Refresher

Session 1 · Wed Aug 26, 2026

Course at a Glance

CSCI 3334 — Systems Programming · Section 03

Fall 2026 · traditional, face-to-face

- **Meets:** Monday & Wednesday, 3:30–4:45 PM
- **Classroom:** Edinburg Campus · EIEAB 2.205
- **Instructor:** Dr. Pu Jiao (JP)
- **Email:** pu.jiao@utrgv.edu

Office hours

- Monday & Wednesday, 2:20–3:20 PM
- EIEAB 3.237
- Or by appointment

Bring the code that is misbehaving. Five minutes in office hours is often more useful than a day of email.

Two Programs That Should Not Surprise You

No definitions yet. First, two short programs. Both are valid C; a compiler may warn about their mixed signed/unsigned comparison. Predict the output before reading on.

```c
int n = -1;
unsigned m = 1u;
printf("%s\n", n < m ? "true" : "false");   /* -1 < 1 ... surely? */
```

Question: what did C change before it compared the two values?

It prints **`false`** on every conforming implementation. The usual arithmetic conversions first convert **`n`** to **`unsigned int`**; **`-1`** becomes **`UINT_MAX`**, which is not less than **`1u`**.

Make the Conversion Visible

CPython's **`ctypes`** can show the conversion C performs before it compares the two values:

```
>>> from ctypes import c_uint
>>> n, m = -1, 1
>>> n < m                          # Python compares two ordinary integers
True
>>> c_uint(n).value                # C converts -1 to unsigned int first
4294967295
>>> c_uint(n).value < m
False
```

`c_uint(-1)` yields **`UINT_MAX`** (for the platform's unsigned-int width). In the C expression, **`int`** and **`unsigned int`** have the same rank and unsigned cannot represent negative **`int`** values, so **`n`** converts to **`unsigned int`**; Python merely makes that conversion visible.

Same Work, Twenty Times the Time

The snippet is abbreviated; see the complete workload.

```
/* complete workload initializes the array and consumes sum */
static int a[4096][4096];                       /* initialized 64 MiB array */

for (i..) for (j..) sum += a[i][j];             /* row-major    */
for (j..) for (i..) sum += a[i][j];             /* column-major */
```

The complete workload initializes the array and consumes the sum so the loops cannot disappear. On an AMD Ryzen 7 8845HS, GCC **–O1** (median of five runs; per traversal):

```
row-major          5.1 ms
column-major     131.8 ms        ~26x slower
```

The source has no cache operations. The explanation lives below the language, and that layer is what this course is about.

Same Instructions, Different Cache Traffic

AMD Ryzen 7 8845HS · Fedora Linux · GCC -O1 · median of 5 runs · 16 passes over a 64 MiB array

Same work

Both orders retired **1.41 billion instructions**. The difference is where each load goes.

Next address

+ 4 B	row-major	next int
+ 16 KiB	column-major	

Profile-build time (16 passes)



26× slower

L2 cache outcomes



17.9M L2 requests · 17K misses



487M L2 requests · 144M misses

AMD PMU counters: l2_cache_req_stat.dc_access_in_l2, dc_hit_in_l2, ls_rd_blk_c. “Hit” = L2 data-cache hit; “miss” = data-cache read miss.

What Is a System?

A **system** is a collection of interacting components that provides useful behavior under constraints. Software systems often serve other programs and manage resources that are **finite** and **shared**.

Each part of that definition does real work:

- **Its users are programs, not people.** Correctness is a contract some other program depends on — not a screen a human can squint at and sanity-check.
- **The resource is finite.** CPU time, physical memory, disk bandwidth, file descriptors, sockets. Running out is a *normal case you must handle*, not an unthinkable disaster.
- **It is shared.** Something has to arbitrate between competing claims. That arbitration *is* the system.

A calculator app is also a system; its memory allocator is a subsystem with a particularly visible resource contract.

Where the Line Falls

What you see	The system underneath it
a web page	the browser's layout engine and JS runtime
a Python script	CPython — bytecode loop, object allocator, the GIL
a to-do app	SQLite — pages, B-trees, write-ahead log, `fsync`
a training script	CUDA driver, GPU allocator, NCCL, the page cache
`printf("hi")`	libc's output buffer and the kernel's write path

The line is not drawn by language or by domain. It is drawn by **who is holding the resource**. Write a program that consumes memory and you are on the left. Write the thing that hands memory out and you are on the right.

The Layer Picture

Each boundary is an interface with a contract:

- **app → library** — a function call. Same process, same address space, no permission needed.
- **library → kernel** — a **system call**. Privilege change, and the kernel is allowed to refuse.
- **kernel → hardware** — instructions, device registers, interrupts.



Two Kinds of Systems Work

- **Implementing** one of the lower boxes — a library, a kernel subsystem, a driver.
- **Debugging a failure that crosses a boundary** — common, and often harder, because a call stack alone may not reveal a contract or environment state.

When a library call faults or a system call returns an error, start with the documented contract: caller preconditions, returned error, and environment. The boundary is a place to investigate, not a presumptive culprit.

What Is Systems Programming?

Systems programming builds software that directly participates in implementing or controlling a computing system. Systems work commonly has some of these traits:

1. **Resources and ownership are explicit.** Memory, files, threads, handles, devices, and other finite resources have observable acquisition, ownership, limits, and release behavior.
2. **Failure is part of the contract.** Operations can fail, block, time out, complete partially, or leave recoverable state. Every outcome must be understood whether represented by an error code, Result, exception, signal, or fault.
3. **Interfaces require exact compatibility.** ABIs, protocols, binary layouts, syscalls, and hardware interfaces must agree on representation and semantics across independently implemented or deployed components.
4. **Operational constraints can be correctness constraints.** Latency, throughput, memory consumption, predictability, power usage, and deadlines may determine whether the system works at all.
5. **Execution and side effects must match the machine model.** Concurrency, memory ordering, interrupts, asynchronous I/O, caching, and hardware behavior cannot be abstracted away when they affect correctness.

The Six Questions This Course Answers

1. How are numbers, addresses, and structures represented as bytes? *(Lectures 2–5b, Lab 1)*
2. What instructions does the CPU actually execute? *(Lectures 6–8, Lab 2)*
3. How does the kernel create, isolate, and schedule processes? *(Lectures 9–11, 17–19, Labs 3 and 5)*
4. How do files and devices move data? *(Lectures 12–13)*
5. Why does *where* data sits change performance? *(Lectures 14–16, Lab 4)*
6. How do multiple threads cooperate without corrupting each other? *(Lectures 20 and 22, Lab 6)*

Six views of one running program — not six unrelated topics. Today's two opening exhibits already belong to questions 1 and 5.

What Systems Programming Is Not

- **Not "writing C."** C is our vehicle because it hides the least. But plenty of C involves no systems thinking at all, and plenty of systems work happens in Rust, Zig, Go, or C++. The ideas transfer; the syntax is incidental.
- **Not micro-optimization.** Hand-writing assembly to save three cycles is not the skill. Recognizing that the loop is limited by memory bandwidth, so those three cycles cannot possibly matter, *is* the skill.
- **Not kernel-only.** Allocators, databases, browsers, language runtimes, and ML frameworks are all systems, all in user space.
- **Not a synonym for embedded or for security.** Both are heavy *consumers* of systems knowledge. Neither is the field.
- **Not legacy archaeology.** Every layer in this course is under active development right now — see the AI segment below.

“Low-Level” Does Not Mean “Fast”

The most common misconception, and worth killing early: **C gives you control, not speed.**

A naive C loop can easily lose to NumPy. Not because Python is fast, but because NumPy’s inner loops are hand-tuned C with cache blocking and SIMD, and yours is neither.

Control is what makes speed *achievable* — you get to choose the data layout, choose when to copy, choose what stays in cache. It is a tool, not a guarantee. Both programs in Exhibit B were C, and one was ~26x slower.

From C to a Running Process

```
hello.c --preprocess--> hello.i --compile--> hello.s --assemble--> hello.o --
link--> hello
```

```
gcc -E hello.c -o hello.i    # macros expanded, #includes pasted inline
gcc -S hello.c -o hello.s    # x86-64 assembly text
gcc -c hello.c -o hello.o    # machine code, symbols still unresolved
gcc hello.o -o hello         # linked against libc into an executable
```

Three things we speak of loosely but must not confuse:

- the **source** — text describing intent
- the **executable** — a file on disk in ELF format
- the **process** — a running instance, with its own memory and its own PID

Live Demo: Proving the Layers Are Real

Using **`src/lab0/hello.c`** — six lines, one **`printf`**. Five commands, each one exposing a different boundary:

```
gcc -std=c17 -Wall -Wextra -Werror -g -O0 -fno-pie -no-pie hello.c -o hello
ldd ./hello               # what else loads, including the loader itself
objdump -d ./hello        # the instructions the CPU will really run
strace ./hello            # every request this program makes of the kernel
ltrace ./hello            # library calls, for contrast with the above
valgrind ./hello          # detects many invalid accesses and leaks
```

Four things to watch for:

1. **`hello.c`** never mentions **`libc.so.6`**, yet it loads.
2. In this GCC build, the constant-string **`printf`** call becomes **`puts`**.
3. With stdout attached to the terminal, this run shows one **`write(fd=1, ...)`**.
4. **`main`** is not the first code that runs.

What the Demo Showed

```
main:
    push rbp                    ; save the caller's frame pointer
    mov  rbp, rsp               ; establish this function's stack frame
    mov  edi, 0x401178          ; argument 1: the address of the string
    call puts@plt               ; enter libc through the dynamic-link table
    mov  eax, 0                 ; return 0 to whoever started us
    pop  rbp
    ret
```

```
hello.c -> gcc -> ELF file -> execve -> ld-linux -> libc puts
        -> stdout buffer -> write(fd=1, ...) -> kernel tty layer -> terminal
```

This representative disassembly is from the non-PIE build above. Instructions, addresses, and library calls vary with compiler, flags, and platform; the layered roles remain.

AI Systems Connection

The claim worth arguing with: *“models write the code now, so the layer below stops mattering.”*

For low-batch decoding of a dense model, a useful first approximation is that a token streams roughly the model’s weights. A 70-billion-parameter model at two bytes per weight moves ~140 GB. On an accelerator with roughly 3 TB/s of memory bandwidth:

```
140 GB / 3 TB/s  ~=  47 ms per token  ~=  21 tokens/sec
```

This is a bandwidth lower bound, not a universal ceiling: batching, parallelism, quantization, reuse, and architecture change it; at larger batches compute can dominate. Quantization reduces weight traffic; KV caching avoids recomputing prior attention keys and values.

Exhibit B again, at datacenter scale: the bottleneck is moving bytes.

Case Study: One 7B Model, Five Different Files

Open **`TheBloke/Llama-2-7B-GGUF`** on the Hugging Face Hub. These files encode the same model architecture and checkpoint at different quantizations. Dividing file size by Llama-2-7B's **6,738,415,616** parameters estimates effective file bits per parameter:

File	On disk	Effective file bits / parameter
fp16 **`.safetensors`**, 2 shards *(base repo)*	13.48 GB	16.00
`llama-2-7b.Q8_0.gguf`	7.16 GB	8.50
`llama-2-7b.Q5_K_M.gguf`	4.78 GB	5.67
`llama-2-7b.Q4_K_M.gguf`	4.08 GB	4.84
`llama-2-7b.Q2_K.gguf`	2.83 GB	3.36

The effective figure includes quantization blocks plus file metadata, alignment, and any tensors stored differently. The format is a data structure, not merely a bag of numbers.

The File Format Is a C Struct

Q8_0 measured **8.50** bits per weight, not 8.00. Here is llama.cpp's actual definition, and the missing half-bit:

```
#define QK8_0 32
typedef struct {
    ggml_half d;        // delta: one fp16 scale for this block
    int8_t  qs[QK8_0];  // quants: 32 weights
} block_q8_0;           // 2 + 32 = 34 bytes
static_assert(sizeof(block_q8_0) == sizeof(ggml_half) + QK8_0,
              "wrong q8_0 block size/padding");
```

```
34 bytes x 8 bits / 32 weights  =  8.5 bits per weight
```

This explains the dominant 8.5-bit block cost; whole-file size also includes metadata, alignment, and other tensors. If **`d`** were a 4-byte **`float`**, the fields alone would total 36 bytes; a different field type changes the format. The assertion guards the binary layout expected by this implementation.

The AI Stack Is Systems All the Way Down

- **`torch.matmul`** names native C++/CUDA work. A crash may occur below Python, even when Python code or arguments triggered it.
- **PagedAttention** applies virtual-memory paging to the KV cache — fixed blocks and a block table; it reduces external fragmentation but can leave internal waste *(Lectures 10–11)*.
- Data loaders can use worker processes and pinned buffers; **`num_workers=0`** runs in the main process *(Lectures 17, 20–23)*.
- Checkpoints interact with buffered I/O and the page cache; durable writes need an appropriate flush or **`fsync`** *(Lectures 12–13)*.



Even the Container Is Systems Work

A `.safetensors` file is: 8 bytes holding a little-endian `uint64` header length, that many bytes of JSON giving each tensor's `dtype`, `shape`, and `data_offsets`, then one contiguous raw buffer.

- Its layout permits efficient `mmap` and lazy paging. When a loader uses that path, first access may fault pages in; allocation, transfer, and compilation can also contribute to first-inference latency *(Lecture 10)*.
- The spec fixes little-endian encoding and C-contiguous tensor data.

Exhibit B's traversal order is written into the file format.

Why This Course, Now

A model can produce plausible C very quickly. Plausible C contains use-after-free, signed overflow, unchecked `malloc`, and off-by-one — and none of those fail the happy-path test you would write to check it.

So the scarce skill moves. Less time spent typing the loop, more spent answering:

- Is this memory freed exactly once, on **every** path, including the error paths?
- Does this depend on undefined behavior that `-O2` is allowed to exploit?
- Is this actually the bottleneck, or does it merely look expensive?

You cannot review what you cannot read. That is the value of the next fourteen weeks.

A Running Thread, Not One Lecture

Every session this term ends with an *AI Systems Connection* segment tying that day's material to how real AI systems behave:

- quantization to integer representation
- KV cache paging to virtual memory
- dataloaders to processes
- collectives to concurrency

The engineering underneath AI is one of the largest employers of exactly this knowledge, and we will keep pointing at it.

Two Halves, One Lecture

Everything above was the *why*: what a system is, why it matters, why the AI stack does not change that. Everything below is the *how* — a fast, targeted pass over the C you already sort of know, tightened to exactly the parts systems work depends on: raw memory, pointer/array mechanics, ownership, and undefined behavior. Lab 0 tests this half directly.

Why We Write C Here

C is the smallest step away from the machine that is still readable. It gives you addresses and bytes directly — and protects you from nothing:

```c
int *p = (int *)0xdeadbeef;   /* compiles without complaint */
*p = 42;                      /* probably crashes */
```

No garbage collector, no bounds checks, no automatic lifetimes. That makes bugs dangerous, and it makes the machine's behavior *observable*. Crashes and leaks are evidence; our job is to read the evidence.

Memory Is One Flat Array of Bytes

On our x86-64 Linux teaching platform, virtual memory behaves like a large byte-addressed region; mappings and permissions matter later. An **address** locates bytes, and a **type** tells C how to interpret them.

```c
int    x = 42;   /* 4 bytes, two's complement on our platform */
double d = 42;   /* 8 bytes, IEEE-754 here                    */
char   c = 42;   /* 1 byte by C definition                    */
```

One value, three different byte patterns. Types do not *protect* bytes, they *interpret* them — which is why a cast changes nothing in memory and everything about meaning.

A Pointer Is an Address with a Type

```
int x = 42;
int *p = &x;    /* &  — "the address of"     */
*p = 99;        /* *  — "go to that address" */
/* x is now 99 */
```

- **`p`** is an ordinary variable; it has an address of its own (**`&p`**).
- **`sizeof(p)`** is 8 on our machines, whatever it points to.
- On our platform, dereferencing **`int *`** accesses a 4-byte **`int`**; **`p + 1`** advances **`sizeof *p`** bytes, not one.

Prompt: after **`*p = 99`**, what changed — **`p`**, **`x`**, or both?

Arrays Are Not Pointers

```
int values[5] = {10, 20, 30, 40, 50};
int *first = values;      /* the array "decays" to &values[0] */

values[2]         /* 30 */
*(first + 2)      /* 30 — the same expression, by definition  */
sizeof(values)    /* 20 — the whole array                     */
sizeof(first)     /* 8  — one pointer                         */
```

`values[i]` is *defined* as **`*(values + i)`**, which is why the ridiculous **`2[values]`** also compiles.

In a function parameter, **`int arr[100]`** adjusts to **`int *arr`**; the **`100`** does not provide a run-time length:

```
void f(int arr[100]) { sizeof(arr); }   /* 8, not 400 — arr is a pointer */
```

Pass or encode the length when the interface needs it.

Stack, Heap, and Ownership

```
int *broken(void) {
    int local = 42;
    return &local;                  /* local dies at return — dangling */
}

int *ok(void) {
    int *heap = malloc(sizeof(int));
    if (!heap) return NULL;         /* check. every time. */
    *heap = 42;
    return heap;                    /* caller now owns this and must free it */
}
```

Returning **`&local`** creates a dangling pointer: this local object's lifetime ends when the function returns. Returning **`heap`** transfers ownership to the caller, who must eventually call **`free`**.

Stack vs Heap

	Stack	Heap
lifetime	automatic-storage lifetime	until its owner releases it
managed by	compiler/runtime	program/allocator
size	limited by the thread stack	allocator and address-space limits
cost	usually low	an allocation call *(Lecture 11)*

Every allocation needs a documented owner. “Who frees this?” is a question your code must be able to answer.

The Four Bugs You Will Meet This Semester

```c
int *p = malloc(64);
free(p);
*p = 1;              /* 1. use-after-free — writes memory you gave back  */
free(p);             /* 2. double free    — undefined behavior */

int *q = malloc(64);
q = malloc(64);      /* 3. leak — the first block is now unreachable */

int a[4];
a[4] = 7;            /* 4. out of bounds — a[4] is not part of a */
```

None of these is required to crash at the bad line. They can fail later or appear to work, possibly only failing in a different run. Valgrind detects many such memory errors; it is useful evidence, not proof of correctness.

Undefined Behavior Is a Contract You Can Break Silently

C's rules are a bargain. You promise not to do certain things; in exchange the compiler optimizes aggressively. Break the promise and it owes you nothing.

```
int i = INT_MAX;
i + 1;       /* UB — not "wraps to INT_MIN"           */
a[4];        /* UB on int a[4] — not "the next variable" */
*(int *)0;   /* UB — not "crashes"                     */
```

"It worked when I tested it" proves only that *this* compiler at *this* optimization level happened to pick something convenient. The same source at **`-O2`** may legally behave differently; Lecture 8 shows the compiler doing it.

House rule: **`-Wall -Wextra -Werror`**, always. A valuable diagnostic, not proof: it diagnoses the signed/unsigned comparison in Exhibit A.

Structs and the Shape of Lab 0

```c
typedef struct node {
    int value;
    struct node *next;   /* must say `struct node` — the typedef isn't ready ye
} Node;
```

```
head -> [ 1 | *-]-> [ 2 | *-]-> [ 3 | NULL ]
```

Each node is a *separate* **`malloc`**. **`sizeof(Node)`** is 16 — it tells you nothing about the list's length. And freeing the head first throws away the only pointer to the rest:

```c
while (head) {
    Node *next = head->next;   /* save it before you free */
    free(head);
    head = next;
}
```

Lab 0 asks for exactly this, plus **`list_append`**, clean under Valgrind.

The Toolchain Is One Debugging Workflow

Tool	The question it answers
`gcc -Wall -Wextra -Werror`	is anything obviously wrong before I even run it?
`make`	did I actually rebuild what I changed?
`gdb`	what is the state *right now*, at this line?
`valgrind`	did it detect a memory access or leak problem?
`git`	what did I change since it last worked?

Debugging is not staring at code and guessing. It is narrowing: form a hypothesis, pick the tool that can *refute* it, run it. Lab 0 walks you through all five.

The Labs Turn Questions into Programs

0. **Warmup** — environment, GDB, a memory bug, a linked list *(bonus points)*
1. **Bit puzzles** — integer and float representation, with restricted operators
2. **Binary mysteries** — read and reverse engineer real x86-64
3. **Cache simulator** — model the memory hierarchy that caused Exhibit B
4. **A shell** — create processes and wire up their I/O
5. **Threads and a mutex** — split work across threads without corrupting it

The goal is never just a program that passes. It is a program you can *explain* — why it works, where it would break, and which tool would show you.

Before the Next Class

1. Set up the Linux development environment (Lab 0, Part 1 — WSL2 instructions included).
2. Clone the Lab 0 repository and run **`make`**.
3. Read CS:APP Chapter 1. Skim §3.1–3.3 if pointers feel rusty.
4. Run today's two exhibits yourself, in **`src/lecture01/`** — then try to explain the numbers you get.

Bring anything that behaved differently from what you expected. The gap between your expectation and the machine's behavior is where this course lives.

Open the Lab 0 handout

Practice I

1. Explain in one sentence why `-1 < 1u` is false. What single change to the declarations makes it true?
2. `sizeof(values)` is 20 but `sizeof(first)` is 8, though both “are” the array. Why is that not a contradiction?
3. Name the resource being managed, and who manages it, for each: `malloc`, the OS scheduler, a CPU cache, SQLite.
4. A function returns `&local`. The program works anyway. Why is it still wrong?

Practice II

5. Column-major traversal did ~26x worse with the same instruction count. Propose a mechanism, and name the experiment that would confirm it.
6. llama.cpp's 4-bit block packs two weights per byte:

```c
typedef struct {
    ggml_half d;             /* 2 bytes */
    uint8_t qs[QK4_0 / 2];   /* QK4_0 is 32 */
} block_q4_0;
```

   Compute its bits per weight. Then predict the size of a 6,738,415,616-weight model stored entirely in this format, and compare your answer to the 4.08 GB that **`Q4_K_M`** actually occupies. What must **`Q4_K_M`** be doing differently?
7. Pick any quantized model on the Hugging Face Hub. Divide its file size by its parameter count to get bits per weight. Does your number match the name of the quantization scheme? Explain the gap.

Reading

- CS:APP Chapter 1 (A Tour of Computer Systems)
- CS:APP §3.1–3.3 (the relationship between C and machine code)
- Lab 0 handout — do Part 1 before the next session

Optional, for the AI segment — skim, do not study:

- The **`safetensors`** format spec (the “Format” section is one page): https://github.com/huggingface/safetensors#format
- Any GGUF model card’s provided-files table, e.g. https://huggingface.co/TheBloke/Llama-2-7B-GGUF — read it as a table of byte-width tradeoffs
- **`ggml/src/ggml-common.h`** in llama.cpp — the quantization block structs. You will be able to read every line of it by Lecture 4.